PYTHON FOR BEGINNERS · SESSION 17

NumPy Operations

Vectorised maths, aggregation, reshaping and random numbers.

Recap

Ask the class:

- What does `a.shape` tell you for a table with 3 rows and 4 columns?
- Is a NumPy slice a **copy** or a **view**?
- What does `a[:, 1]` give for a 2D array?

Today: do **maths** on whole arrays, **summarise** them, **reshape** and join them, and make **random** numbers.

Vectorised maths

Arithmetic on an array happens on **every item at once**. No loop needed.

```
import numpy as np

a = np.array([10, 20, 30, 40])

print(a + 5)
print(a - 5)
print(a * 3)
print(a / 4)
print(a // 3)
print(a % 3)
print(a ** 2)
```

OUTPUT

```
[15 25 35 45]
[ 5 15 25 35]
[ 30  60  90 120]
[ 2.5  5.   7.5 10. ]
[ 3  6 10 13]
[1 2 0 1]
[ 100  400  900 1600]
```

Two arrays of the **same shape** work **item by item**.

```
import numpy as np

price = np.array([100, 250, 80])
qty = np.array([2, 1, 5])

cost = price * qty
print(cost)
print(cost.sum())
```

OUTPUT

```
[200 250 400]
850
```

Maths functions

FUNCTION	WHAT IT DOES
`np.sqrt(a)`	square root of each item
`np.abs(a)`	remove the minus sign
`np.exp(a)`	e to the power of each item
`np.log(a)`	natural logarithm
`np.sin(a)`, `np.cos(a)`	trigonometry (angles in radians)
`np.round(a, 2)`	round to 2 decimal places
`np.floor(a)`, `np.ceil(a)`	round down / up

```
import numpy as np

a = np.array([1, 4, 9, 16])
print(np.sqrt(a))

b = np.array([-3, 2, -7])
print(np.abs(b))

c = np.array([1.234, 5.678, 9.1011])
print(np.round(c, 2))
print(np.floor(c))
print(np.ceil(c))

print(np.exp(np.array([0, 1])))
print(np.log(np.array([1, 10])))
```

OUTPUT

```
[1. 2. 3. 4.]
[3 2 7]
[1.23 5.68 9.1 ]
[1. 5. 9.]
[ 2.  6. 10.]
[1.         2.71828183]
[0.         2.30258509]
```

Use np.sqrt, not math.sqrt

`math.sqrt` works on **one** number. `np.sqrt` works on a **whole array**. Mixing them up gives a `TypeError`.

Examples of vectorised maths

Celsius to Fahrenheit

```
import numpy as np

celsius = np.array([0, 25, 37, 100])
fahrenheit = celsius * 9 / 5 + 32
print(fahrenheit)
```

OUTPUT

```
[ 32.   77.   98.6 212. ]
```

Add GST to every price

```
import numpy as np

prices = np.array([100, 250, 80, 400])
with_gst = np.round(prices * 1.18, 2)
print(with_gst)
print("Bill total:", with_gst.sum())
```

```
OUTPUT
[118.  295.   94.4 472. ]
Bill total: 979.4
```

Distance between points

The distance from the origin to the points `(3, 4)`, `(6, 8)` and `(5, 12)`:

```
import numpy as np

x = np.array([3, 6, 5])
y = np.array([4, 8, 12])

distance = np.sqrt(x ** 2 + y ** 2)
print(distance)
```

```
OUTPUT
[ 5. 10. 13.]
```

Is it really faster?

```
import numpy as np
import time

n = 300000
data = list(range(n))
arr = np.arange(n)

t0 = time.time()
slow = [x * 2 for x in data]
t_list = time.time() - t0

t0 = time.time()
fast = arr * 2
t_numpy = time.time() - t0

print("Same answer:", slow[-1] == fast[-1])
print("NumPy was faster:", t_numpy < t_list)
```

```
OUTPUT
Same answer: True
NumPy was faster: True
```

The exact times change from computer to computer, but NumPy is usually **many times faster** on big data.

Broadcasting

Broadcasting is how NumPy combines arrays of **different shapes**. The smaller one is **stretched** to fit the bigger one.

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a + 10)

row = np.array([100, 200, 300])
print(a + row)

col = np.array([[10],
                [20]])
print(a + col)
```

OUTPUT

```
[[11 12 13]
 [14 15 16]]
[[101 202 303]
 [104 205 306]]
[[11 12 13]
 [24 25 26]]
```

- `a + 10`: the single number is used for **every** item.
- `a + row`: the row `[100 200 300]` is added to **each row**.
- `a + col`: the column is added to **each column** (the `10` goes across row 0, the `20` across row 1).

The rule

Compare the shapes from the **right-hand end**. Two sizes fit if they are **equal**, or one of them is **1**.

SHAPES	WORKS?	WHY
`(2, 3)` and `(3,)`	yes	3 matches 3; the missing size is stretched
`(2, 3)` and `(2, 1)`	yes	the 1 is stretched to 3
`(2, 3)` and `(2,)`	**no**	3 does not match 2
`(3,)` and `(4,)`	**no**	3 does not match 4

Try it: this fails on purpose

```
import numpy as np

a = np.ones((2, 3))
b = np.array([1, 2])
print(a + b)
```

ERROR

```
  File "your code", line 5, in <module>
ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

Use it: subtract each subject's average

```
import numpy as np

marks = np.array([[80, 70, 90],
                  [60, 90, 75],
                  [70, 80, 60]])

avg = marks.mean(axis=0)
print(avg)
print(marks - avg)
```

OUTPUT

```
[70. 80. 75.]
[[ 10. -10.  15.]
 [-10.  10.   0.]
 [  0.   0. -15.]]
```

Conditions: where, clip, any, all

`np.where(condition, a, b)` picks `a` where the condition is `True` and `b` where it is `False`. It is a vectorised `if` and `else`.

```
import numpy as np

marks = np.array([45, 82, 67, 91, 38, 74])

result = np.where(marks >= 50, "Pass", "Fail")
print(result)

bonus = np.where(marks < 50, marks + 5, marks)
print(bonus)
```

```
OUTPUT
['Fail' 'Pass' 'Pass' 'Pass' 'Fail' 'Pass']
[50 82 67 91 43 74]
```

Grades with more than two choices

```
import numpy as np

marks = np.array([45, 82, 67, 91, 38, 74])

grade = np.where(marks >= 80, "A",
         np.where(marks >= 60, "B",
          np.where(marks >= 40, "C", "D")))
print(grade)
```

```
OUTPUT
['C' 'A' 'B' 'A' 'D' 'B']
```

clip: keep numbers in a range

```
import numpy as np

a = np.array([-5, 3, 12, 7, 20])
print(np.clip(a, 0, 10))
```

```
OUTPUT
[ 0  3 10  7 10]
```

Anything below `0` becomes `0`, and anything above `10` becomes `10`.

any, all and counting

```
import numpy as np

marks = np.array([45, 82, 67, 91, 38, 74])

print((marks > 90).any())
print((marks > 30).all())
print((marks >= 50).sum())
print(np.count_nonzero(marks < 50))
```

OUTPUT

```
True
True
4
2
```

- `.any()` is `True` if **at least one** item matches.
- `.all()` is `True` if **every** item matches.
- `.sum()` on `True` / `False` counts the matches, because `True` is `1`.

Aggregation

Aggregation squeezes many numbers into **one** (or a few). You met `sum` and `mean`. Here are the rest.

FUNCTION	WHAT IT GIVES
`a.sum()`	total
`a.mean()`	average
`np.median(a)`	the middle value
`a.min()`, `a.max()`	smallest, biggest
`a.std()`	standard deviation (how spread out)
`a.var()`	variance (the square of `std`)
`a.prod()`	everything multiplied together
`np.percentile(a, 90)`	the value 90 percent of items are below

```python
import numpy as np

marks = np.array([45, 82, 67, 91, 38, 74, 88, 59])

print("sum   :", marks.sum())
print("mean  :", marks.mean())
print("median:", np.median(marks))
print("min   :", marks.min())
print("max   :", marks.max())
print("range :", marks.max() - marks.min())
print("std   :", round(marks.std(), 2))
print("90th percentile:", np.percentile(marks, 90))
```

OUTPUT

```
sum   : 544
mean  : 68.0
median: 70.5
min   : 38
max   : 91
range : 53
std   : 18.28
90th percentile: 88.9
```

Mean or median?

One huge number pulls the **mean** up, but barely moves the **median**. For `[1, 2, 3, 4, 1000]` the mean is `202` and the median is `3`.

```python
import numpy as np

a = np.array([1, 2, 3, 4, 1000])
print(a.mean())
print(np.median(a))
```

OUTPUT

```
202.0
3.0
```

Aggregation along an axis

On a table, `axis=0` gives one answer **per column** and `axis=1` gives one answer **per row**.

```
import numpy as np

marks = np.array([[85, 90, 78],
                  [70, 65, 80],
                  [92, 88, 95],
                  [55, 60, 48]])

print(marks.sum(axis=0))
print(marks.sum(axis=1))
print(marks.mean(axis=0))
print(marks.max(axis=1))
print(marks.argmax(axis=0))
```

OUTPUT

```
[302 303 301]
[253 215 275 163]
[75.5  75.75 75.25]
[90 80 95 60]
[2 0 2]
```

- `sum(axis=0)`: the total of **each subject** (column).
- `sum(axis=1)`: the total of **each student** (row).
- `argmax(axis=0)`: which **student** (row number) has the top mark in each subject.

keepdims

`keepdims=True` keeps the result as a table, so it can be used in more maths.

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

totals = a.sum(axis=1, keepdims=True)
print(totals.shape)
print(a / totals)
```

OUTPUT

```
(2, 1)
[[0.16666667 0.33333333 0.5       ]
 [0.26666667 0.33333333 0.4       ]]
```

Each item is divided by its **row total**, so every row now adds up to `1`.

Cumulative maths and unique values

cumsum, cumprod and diff

```
import numpy as np

daily = np.array([5, 3, 8, 2, 7])

print(np.cumsum(daily))
print(np.cumprod(np.array([1, 2, 3, 4])))
print(np.diff(daily))
```

```
OUTPUT
[ 5  8 16 18 25]
[ 1  2  6 24]
[-2  5 -6  5]
```

- `cumsum` is a **running total**: `5, 5+3, 5+3+8 ...`
- `cumprod` is a running product.
- `diff` is the **change** from one item to the next. It is one item shorter.

sort and argsort

```
import numpy as np

marks = np.array([45, 82, 67, 91, 38])

print(np.sort(marks))
print(np.sort(marks)[::-1])
print(np.argsort(marks))
```

```
OUTPUT
[38 45 67 82 91]
[91 82 67 45 38]
[4 0 2 1 3]
```

`argsort` gives the **positions** that would sort the array. The first one, `4`, is the position of the smallest mark (`38`).

unique and counting

```
import numpy as np

colours = np.array(["red", "blue", "red", "green", "blue", "red"])

names, counts = np.unique(colours, return_counts=True)
print(names)
print(counts)
```

OUTPUT

```
['blue' 'green' 'red']
[2 1 3]
```

Reshaping

Last class: `reshape`, `T` and `flatten`. A quick recap, then new tools.

```
import numpy as np

a = np.arange(1, 13)
print(a.reshape(3, 4))
print(a.reshape(2, -1).shape)
print(a.reshape(3, 4).T.shape)
print(a.reshape(3, 4).ravel())
```

OUTPUT

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
(2, 6)
(4, 3)
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

NAME	WHAT IT DOES
`ravel()`	flatten to 1D (a view when it can, so it is quick)
`flatten()`	flatten to 1D, always a **copy**
`np.expand_dims(a, axis)`	add a new dimension of size 1
`a[:, np.newaxis]`	also adds a dimension: row to column
`np.squeeze(a)`	remove dimensions of size 1

Rows into columns

```
import numpy as np

a = np.array([1, 2, 3])
print(a.shape)

col = a[:, np.newaxis]
print(col)
print(col.shape)

row = col.reshape(1, 3)
print(row.shape)

print(np.squeeze(row).shape)
```

```
OUTPUT
(3,)
[[1]
 [2]
 [3]]
(3, 1)
(1, 3)
(3,)
```

A **1D** array `(3,)` is neither a row nor a column. `newaxis` turns it into a **column** `(3, 1)`, which broadcasting can then use.

```
import numpy as np

a = np.array([1, 2, 3])
col = np.array([10, 20, 30])[:, np.newaxis]

print(a + col)
```

```
OUTPUT
[[11 12 13]
 [21 22 23]
 [31 32 33]]
```

Every item of `a` is added to every item of `col`. This makes a **table of all the sums**, like a times table.

Joining and splitting arrays

FUNCTION	WHAT IT DOES
`np.concatenate([a, b])`	join end to end (choose the `axis`)
`np.vstack([a, b])`	stack **vertically**: add more rows
`np.hstack([a, b])`	stack **horizontally**: add more columns
`np.split(a, 3)`	cut into 3 equal parts

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(np.concatenate([a, b]))
print(np.vstack([a, b]))
print(np.hstack([a, b]))
```

OUTPUT

```
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
[1 2 3 4 5 6]
```

```
import numpy as np

t1 = np.array([[1, 2],
               [3, 4]])
t2 = np.array([[5, 6],
               [7, 8]])

print(np.concatenate([t1, t2], axis=0))
print(np.concatenate([t1, t2], axis=1))
```

OUTPUT

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
[[1 2 5 6]
 [3 4 7 8]]
```

- `axis=0` adds the second table **below** the first (more rows).
- `axis=1` adds it **beside** the first (more columns).
- The sizes of the **other** direction must match.

Adding a total column

```
import numpy as np

marks = np.array([[85, 90, 78],
                  [70, 65, 80]])

totals = marks.sum(axis=1, keepdims=True)
print(np.hstack([marks, totals]))
```

```
OUTPUT
[[ 85  90  78 253]
 [ 70  65  80 215]]
```

split

```
import numpy as np

a = np.arange(1, 10)
parts = np.split(a, 3)
print(parts)
print(len(parts))

for p in parts:
    print(p.sum())
```

```
OUTPUT
[array([1, 2, 3]), array([4, 5, 6]), array([7, 8, 9])]
3
6
15
24
```

Random numbers

NumPy can make a **whole array** of random numbers in one go, and much faster than the `random` module in a loop.

The modern way is to make a **generator** with `np.random.default_rng()` and ask it for numbers.

METHOD	WHAT IT GIVES
`rng.random(size)`	floats from `0.0` up to (not including) `1.0`
`rng.integers(low, high, size)`	whole numbers from `low` up to (not including) `high`
`rng.normal(mean, std, size)`	bell-curve numbers around the mean
`rng.choice(items, size)`	pick items at random
`rng.shuffle(a)`	mix an array in place
`rng.permutation(a)`	a shuffled **copy**

The numbers change every run, so this box prints **different** numbers each time you press **Run**.

```
import numpy as np

rng = np.random.default_rng()

print(rng.random(3))
print(rng.integers(1, 7, size=5))
print(rng.random((2, 3)).shape)
```

```
OUTPUT
[0.61787815 0.73794018 0.71356958]
[1 4 4 6 2]
(2, 3)
```

integers excludes the top

Unlike `random.randint(1, 6)`, `rng.integers(1, 7)` gives `1` to `6`. The `high` value is **not** included. Roll a die with `integers(1, 7)`.

A seed repeats the numbers

```
import numpy as np

a = np.random.default_rng(42).random(4)
b = np.random.default_rng(42).random(4)
c = np.random.default_rng(7).random(4)

print(np.array_equal(a, b))
print(np.array_equal(a, c))
```

```
OUTPUT
True
False
```

The **same seed** gives the **same numbers**. That is how a data scientist makes an experiment **repeatable**.

Examples with random numbers

Roll a die 6000 times

```
import numpy as np

rng = np.random.default_rng()
rolls = rng.integers(1, 7, size=6000)

print(rolls.min(), rolls.max())
print(np.bincount(rolls)[1:])
```

ERROR

```
1 6
  File "your code", line 7, in <module>
TypeError: Cannot cast array data from dtype('int64') to dtype('int32') according to the rule
'safe'
```

`bincount` counts how many times each number appears. Each face should come up about **1000** times.

Random marks table

```
import numpy as np

rng = np.random.default_rng(1)
marks = rng.integers(35, 101, size=(4, 3))

print(marks.shape)
print(marks.min() >= 35, marks.max() <= 100)
print(marks.mean(axis=0).shape)
```

OUTPUT

```
(4, 3)
True True
(3,)
```

Pick winners without repeats

```
import numpy as np

names = np.array(["Aman", "Ankit", "Neha", "Rohit", "Sara", "Tara"])
rng = np.random.default_rng()

winners = rng.choice(names, size=3, replace=False)
print(winners)
print(len(set(winners)))
```

```
OUTPUT
['Tara' 'Neha' 'Aman']
3
```

`replace=False` means the same name **cannot** be picked twice.

Bell-curve numbers

```
import numpy as np

rng = np.random.default_rng()
heights = rng.normal(170, 10, size=10000)

print(round(heights.mean()))
print(round(heights.std()))
```

```
OUTPUT
170
10
```

The mean is close to `170` and the standard deviation close to `10`, as we asked.

The older way

You will see old code that uses `np.random.seed(5)`, `np.random.rand(3)` and `np.random.randint(1, 7, 5)`. They still work, but a **generator** (`default_rng`) is the recommended way today.

```
import numpy as np

np.random.seed(5)
x = np.random.rand(3)

np.random.seed(5)
y = np.random.rand(3)

print(np.array_equal(x, y))
print(np.random.randint(1, 7, 5).shape)
```

OUTPUT

```
True
(5,)
```

Example: marks analysis

Put it all together. Rows are students, columns are maths, science and english.

```
import numpy as np

names = np.array(["Aman", "Ankit", "Neha", "Rohit"])
marks = np.array([[85, 90, 78],
                  [70, 65, 80],
                  [92, 88, 95],
                  [55, 60, 48]])

total = marks.sum(axis=1)
average = marks.mean(axis=1)
grade = np.where(average >= 80, "A", np.where(average >= 60, "B", "C"))
rank = np.argsort(-total)

print("Totals :", total)
print("Average:", np.round(average, 1))
print("Grades :", grade)
print("Best to worst:", names[rank])
print("Hardest subject (lowest mean):", ["maths", "science", "english"]
[marks.mean(axis=0).argmin()])
```

OUTPUT

```
Totals : [253 215 275 163]
Average: [84.3 71.7 91.7 54.3]
Grades : ['A' 'B' 'A' 'C']
Best to worst: ['Neha' 'Aman' 'Ankit' 'Rohit']
Hardest subject (lowest mean): english
```

- `np.where` makes the grades for everyone at once.

- `argsort(-total)` sorts **biggest first** (the minus flips the order).
- `names[rank]` uses the positions to **reorder the names**: fancy indexing.

Standardising marks

A **z-score** says how many standard deviations a mark is from the average. It lets you compare marks from different subjects.

```
import numpy as np

marks = np.array([[85, 90, 78],
                  [70, 65, 80],
                  [92, 88, 95],
                  [55, 60, 48]])

z = (marks - marks.mean(axis=0)) / marks.std(axis=0)
print(np.round(z, 2))
print(np.allclose(z.mean(axis=0), 0))
```

```
OUTPUT
[[ 0.67  1.06  0.16]
 [-0.39 -0.8   0.28]
 [ 1.16  0.92  1.16]
 [-1.44 -1.18 -1.6 ]]
True
```

Broadcasting does the work: the column averages are subtracted from **every row**. After standardising, each column has a mean of `0`, and `allclose` confirms it (`True`).

Your turn: print each student's percentage (out of 300), rounded to 1 decimal place

```
import numpy as np

marks = np.array([[85, 90, 78],
                  [70, 65, 80]])
```

Common mistakes

- **Using `math.sqrt` on an array.** Use `np.sqrt`.
- **Using `and` / `or` instead of `&` / `|`** in array conditions.
- **Mixing up `axis=0` and `axis=1`.** `axis=0` goes **down** (one answer per column); `axis=1` goes **across** (one answer per row).
- **Forgetting that `integers(1, 6)` stops at 5.** Use `integers(1, 7)` for a die.
- **Expecting `rng.shuffle(a)` to return something.** It changes `a` in place and returns `None`.
- **Comparing shapes that do not fit.** `(2, 3)` and `(2,)` cannot broadcast.

- **Forgetting the extra brackets in `np.concatenate([a, b])`.** The arrays go in a **list**.
- **Expecting random numbers to repeat.** Use a **seed** when you need the same numbers again.

Try it: this fails on purpose

```
import numpy as np
import math

a = np.array([4, 9, 16])
print(math.sqrt(a))
```

```
ERROR
  File "your code", line 5, in <module>
TypeError: only length-1 arrays can be converted to Python scalars
```

Try it: this also fails on purpose

```
import numpy as np

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.concatenate(a, b))
```

```
ERROR
  File "your code", line 5, in <module>
TypeError: only integer scalar arrays can be converted to a scalar index
```

Practice

Your turn: turn the prices into prices after a 10 percent discount, rounded to 2 places

```
import numpy as np

prices = np.array([199.99, 49.5, 1200, 75.25])
```

Your turn: use np.where to print "even" or "odd" for each number

```
import numpy as np

nums = np.arange(1, 9)
```

Your turn: print the running total of the daily sales

```
import numpy as np

sales = np.array([120, 80, 150, 95, 200])
```

Your turn: make 10 random whole numbers from 1 to 100 and print the biggest

```
import numpy as np

rng = np.random.default_rng()
```

Homework and wrap-up

- Make an array of the temperatures of a week in Celsius and print them in **Fahrenheit** and **Kelvin**.
- Roll **two dice 10 000 times** with NumPy and count how many times the total is **7**.
- Create a `5 x 4` marks table with random whole numbers from 35 to 100. Print each student's **average**, each subject's **highest** mark, and the student with the **best total**.
- Normalise an array so its smallest value becomes `0` and its biggest becomes `1`: `(a - a.min()) / (a.max() - a.min())`.
- Use `np.where` and `np.clip` to cap all values above 100 at 100 and replace negative values with 0.
- Join two tables with `vstack` and `hstack`, and print the shape of each result.
- Revise: broadcasting, `where`, aggregation with `axis`, `cumsum`, `concatenate`, `default_rng`.

Next class

Pandas basics: Series and DataFrame.